

Python3 Request playbook

What Is a Playbook Course?

- Emphasizes real-world examples and workflows.
- Provides reusable patterns and solutions.
- Helps developers solve common tasks quickly.
- Designed for hands-on learning.

World Wide Web (WWW)

- The World Wide Web is a collection of interconnected web resources.
- Websites are accessed through web browsers.
- Communication occurs using HTTP and HTTPS protocols.
- Resources are identified using URLs.
- Servers store and deliver web content.

HTTP

- HTTP stands for HyperText Transfer Protocol.
- It is the foundation of communication on the web.
- Defines how clients and servers exchange data.
- Uses a request-response communication model.
- Supports multiple request methods such as GET and POST.

HTTP Methods

- GET retrieves data from a server.
- POST sends new data to a server.
- PUT updates existing resources.
- DELETE removes resources from a server.
- PATCH partially updates existing resources.

```
import requests

response = requests.get("https://api.github.com")

print(response.status_code)
```

Client-Server Architecture

- A client sends requests to a server.
- The server processes requests and returns responses.
- Browsers, mobile apps, and scripts act as clients.
- Servers store and manage resources.
- HTTP enables communication between clients and servers.

```
import requests

response = requests.get("https://httpbin.org/get")

print(response.text)
```

Request-Response Cycle

- A client initiates communication by sending a request.
- The request contains a URL, headers, and optional data.
- The server processes the request.
- The server returns a response.
- The client interprets and displays the response.

```
import requests

response = requests.get("https://httpbin.org/get")

print(response.status_code)

print(response.headers)
```

HTTP Request

- A request asks a server to perform an action.
- Contains a method such as GET or POST.

- Includes request headers and optional body data.
- Sent by a client application.
- Initiates the request-response cycle.

```
import requests  
  
response = requests.get(  
    "https://httpbin.org/get",  
    params={"name": "John"}  
)  
  
print(response.url)
```

HTTP Response

- Returned by the server after processing a request.
- Contains status code, headers, and content.
- Indicates whether the request succeeded or failed.
- Can return HTML, JSON, images, or files.
- Provides information about the requested resource.

```
import requests  
  
response = requests.get("https://httpbin.org/get")  
  
print(response.status_code)  
  
print(response.text)
```

Status Codes

- 200 indicates a successful request.
- 201 indicates successful resource creation.
- 404 indicates a resource was not found.
- 500 indicates an internal server error.
- Status codes help diagnose request outcomes.

```
import requests

response = requests.get("https://httpbin.org/status/200")

print(response.status_code)
```

Application Programming Interfaces (APIs)

- APIs allow applications to communicate with each other.
- Most web APIs use HTTP requests and responses.
- APIs provide structured access to data and services.
- JSON is the most common API response format.
- Requests library is widely used for API interaction.

```
import requests

response = requests.get(

    "https://jsonplaceholder.typicode.com/users")

data = response.json()

print(data[0]["name"])
```

Requests Library

- Requests is a popular Python HTTP library.
- Simplifies sending HTTP requests.
- Supports GET, POST, PUT, DELETE, and PATCH methods.
- Handles headers, parameters, cookies, and authentication.
- Provides easy access to response data.

```
import requests

response = requests.get("https://httpbin.org/get")

print(response.status_code)

print(response.text)
```

Working with JSON Responses

- APIs commonly return JSON data.
- Response objects provide a `json()` method.
- JSON data is converted into Python dictionaries and lists.
- Simplifies data extraction and processing.
- Frequently used in API integrations.

```
import requests

response = requests.get(

    "https://jsonplaceholder.typicode.com/posts/1"

)

data = response.json()

print(data["title"])
```

Analyzing GET Requests

- GET requests are used to retrieve data from a server.
- They do not modify server resources.
- Data is sent through the URL as query parameters.
- GET requests are commonly used for searching and fetching records.
- Supported directly by the Requests library.

```
import requests

response = requests.get(

    "https://jsonplaceholder.typicode.com/posts"

)

print(response.status_code)
```

Characteristics of GET Requests

- GET requests are idempotent.
- Repeated requests produce the same result.

- Suitable for read-only operations.
- Can be cached by browsers and servers.
- Can be bookmarked and shared using URLs.

```
import requests  
  
response = requests.get(  
    "https://httpbin.org/get",  
    params={"page": 1}  
)  
  
print(response.url)
```

GET Request Workflow

- Client sends a GET request to a server.
- Request contains a URL and optional parameters.
- Server processes the request.
- Server returns a response.
- Client reads and processes the response data.

```
import requests  
  
response = requests.get("https://httpbin.org/get")  
  
print(response.text)
```

Response Object

- Returned by every Requests library call.
- Contains server response information.
- Stores status code, headers, cookies, and content.
- Provides methods for handling JSON data.
- Used to inspect request results.

```
import requests
```

```
response = requests.get("https://httpbin.org/get")  
print(response.status_code)  
print(response.headers)
```

Response Status Codes

- Indicate whether a request succeeded or failed.
- 200 means success.
- 404 means resource not found.
- 401 means unauthorized access.
- 500 means server error.

```
import requests  
response = requests.get("https://httpbin.org/status/200")  
print(response.status_code)
```

Response Content

- Response body contains returned data.
- `text` returns content as a string.
- `content` returns raw bytes.
- `json()` converts JSON responses into Python objects.
- Frequently used when working with APIs.

```
import requests  
response = requests.get(  
    "https://jsonplaceholder.typicode.com/users/1"  
)  
print(response.json())
```

Cookies

- Cookies store information between requests.

- Sent by the server using the Set-Cookie header.
- Automatically returned by the client in future requests.
- Commonly used for authentication and tracking.
- Help maintain user state.

```
import requests
```

```
response = requests.get("https://httpbin.org/cookies/set/user/John")
```

```
print(response.cookies)
```

Sessions

- Sessions persist settings across multiple requests.
- Automatically manage cookies.
- Improve performance through connection reuse.
- Useful for login-protected websites.
- Reduce repetitive authentication requests.

```
import requests
```

```
session = requests.Session()
```

```
response = session.get("https://httpbin.org/cookies/set/user/John")
```

```
print(session.cookies.get_dict())
```

Session Workflow

- User logs in through a POST request.
- Server returns a session cookie.
- Browser or client stores the cookie.
- Future requests include the cookie automatically.
- Server identifies the authenticated user.

```
import requests
```

```
session = requests.Session()
```

```
login_data = {  
    "username": "admin",  
    "password": "secret"  
}  
  
session.post("https://httpbin.org/post", data=login_data)
```

Authenticating Requests

- Many APIs require authentication.
- Authentication verifies client identity.
- Credentials are sent securely.
- Prevents unauthorized access.
- Often uses API keys or OAuth.

```
import requests  
  
headers = {"Authorization": "Bearer ACCESS_TOKEN"}  
  
response = requests.get("https://api.example.com/data",  
    headers=headers)
```

OAuth 2.0

- Industry-standard authorization framework.
- Allows applications to access user resources securely.
- Uses access tokens instead of passwords.
- Supports delegated access.
- Commonly used by Google, GitHub, and Microsoft APIs.

OAuth 2.0 Components

- Resource Owner is the user.
- Client Application requests access.
- Authorization Server issues tokens.

- Resource Server hosts protected resources.
- Access Tokens authorize API requests.

OAuth 2.0 Authorization Flow

- Application registers with the provider.
- Receives Client ID and Client Secret.
- User grants permission through a consent screen.
- Authorization server returns an authorization code.
- Code is exchanged for an access token.

```
authorization_url = (  
    "https://accounts.google.com/o/oauth2/auth"  
)
```

Access Tokens

- Represent granted permissions.
- Sent with API requests.
- Have limited lifetimes.
- More secure than sharing passwords.
- Required for protected resources.

```
headers = {  
    "Authorization": "Bearer ACCESS_TOKEN"  
}
```

HTTPS

- HTTPS is the secure version of HTTP.
- Encrypts communication between client and server.
- Protects sensitive information.
- Uses SSL/TLS certificates.

- Required by most modern APIs.

```
import requests  
response = requests.get(  
    "https://www.google.com"  
)
```

SSL/TLS Certificates

- Verify server identity.
- Enable encrypted communication.
- Issued by trusted Certificate Authorities (CAs).
- Prevent impersonation attacks.
- Automatically verified by Requests.

```
import requests  
response = requests.get( "https://www.weather.com/my-town")
```

Certificate Verification

- Requests verifies certificates by default.
- Failed verification raises an `SSLError`.
- Ensures secure communication.
- Prevents man-in-the-middle attacks.
- Recommended for production systems.

```
import requests  
response = requests.get(  
    "https://www.weather.com/my-town",  
    verify=True  
)
```

Disabling SSL Verification

- Can bypass certificate validation.
- Useful only for testing environments.
- Not recommended in production.
- Reduces communication security.
- Increases security risks.

```
import requests
```

```
response = requests.get("https://www.weather.com/my-town",verify=False)
```

Custom CA Bundles

- Used when working with private certificates.
- Specifies trusted certificate locations.
- Enables verification with custom authorities.
- Common in enterprise environments.
- Supported through the verify parameter.

```
import requests
```

```
response = requests.get("https://example.com",verify="/path/to/ca_bundle.pem")
```

Client Certificates

- Some servers require client authentication.
- Client provides certificate and private key.
- Adds an extra security layer.
- Common in enterprise APIs.
- Supported directly by Requests.

```
import requests
```

```
response = requests.get(
```

```
    "https://example.com",
```

```
    cert=("client.crt", "client.key"))
```